

CFD 2025-26 – Lab 06 – solution

The full code for this lab is provided in the notebook `CFDlab06.ipynb`. One step of the Euler method is implemented in the function `ex1_assemble_Euler`, by constructing the linear variational problem `vpb` (returned together with the finite element solution `wh` and possibly the nullspace `nsp` depending on the input flag `get_nsp`). If a preconditioning form is provided through the optional input argument `create_pc_form`, then `vpb` uses it.

Exercise 1

- 1 Let us partition the time interval with a timestep Δt , and denote by $t^n = n \Delta t, n = 0, \dots, M = \lceil \frac{1}{\Delta t} \rceil = \lceil \frac{T}{\Delta t} \rceil$ the nodes of the time grid. We can then introduce the approximations $\mathbf{u}^n, p^n, n = 0, \dots, M$ of $\mathbf{u}, p$, respectively (and the same notation is introduced for all other functions, such as $\mathbf{f}^{n+1} = \mathbf{f}|_{t=t^{n+1}}$. The forward Euler method can thus be formulated (in strong form) as:

Given $\mathbf{u}^0 = \mathbf{0}$, for each $n = 0, \dots, M - 1$ find $(\mathbf{u}^{n+1}, p^{n+1})$ such that

$$\begin{cases} \frac{\mathbf{u}^{n+1} - \mathbf{u}^n}{\Delta t} + \nabla p^{n+1} = \mathbf{f}^{n+1} - (\mathbf{u}^n \cdot \nabla) \mathbf{u}^n + \frac{1}{\text{Re}} \Delta \mathbf{u}^n, & \text{in } \Omega, \\ \text{div } \mathbf{u}^{n+1} = 0, & \text{in } \Omega, \\ \mathbf{u}^{n+1} = \mathbf{u}_{\text{ex}}^{n+1}, & \text{on } \partial\Omega. \end{cases} \quad (1)$$

Introducing, for each $n = 0, \dots, M$, the spaces

$$V_D^n = \{\mathbf{v} \in [H^1(\Omega)]^2 : \mathbf{v} = \mathbf{u}_{\text{ex}}^n \text{ on } \partial\Omega\}, \quad V_0 = [H_0^1(\Omega)]^2, \quad Q = L^2_0(\Omega),$$

then a mesh $\mathcal{T}_h$ and the corresponding finite $\mathbb{P}^2 - \mathbb{P}^1$ finite element spaces $V_{D,h}^n \subset V_D^n, V_{0,h} \subset V_0, Q_h \subset Q$, and finally a discrete time-dependent lifting $\mathcal{R}_h^n \mathbf{u}_{\text{ex}} \in V_D^n$ such that $\mathcal{R}_h^n \mathbf{u}_{\text{ex}}(\mathbf{x}) = \Pi_h^2 \mathbf{u}_{\text{ex}}(\mathbf{x}, t^n)$ on $\partial\Omega$, we can derive the weak formulation of the time-advancing problem (3) and the following fully discrete scheme:

Given $\mathbf{u}_h^0 \simeq \mathbf{u}^0 = \mathbf{0}$, for each $n = 0, \dots, M - 1$, find $(\mathbf{u}_h^{n+1}, p_h^{n+1}) \in V_{D,h}^{n+1} \times Q_h$, where $\mathbf{u}_h^{n+1} = \tilde{\mathbf{u}}_h^{n+1} + \mathcal{R}_h^{n+1} \mathbf{u}_{\text{ex}}$ and $\tilde{\mathbf{u}}_h^{n+1} \in V_{0,h}$, such that

$$m(\tilde{\mathbf{u}}_h^{n+1}, \mathbf{v}_h) + b(\mathbf{v}_h, p_h^{n+1}) + b(\tilde{\mathbf{u}}_h^{n+1}, q_h) = F_{\text{FE}}^n(\mathbf{v}_h, q_h) \quad \forall (\mathbf{v}_h, q_h) \in V_{0,h} \times Q_h, \quad (2)$$

where

$$\begin{aligned} m(\mathbf{u}, \mathbf{v}) &= \left(\frac{1}{\Delta t} \mathbf{u}, \mathbf{v} \right), \\ b(\mathbf{v}, q) &= -(\text{div } \mathbf{v}, q), \end{aligned}$$

$$\begin{aligned} F_{\text{FE}}^n(\mathbf{v}, q) &= m(\mathbf{u}_h^n, \mathbf{v}) + (\mathbf{f}^n, \mathbf{v}) - ((\mathbf{u}_h^n \cdot \nabla) \mathbf{u}_h^n, \mathbf{v}) - \frac{1}{\text{Re}} (\nabla \mathbf{u}_h^n, \nabla \mathbf{v}) \\ &\quad - m(\mathcal{R}_h^{n+1} \mathbf{u}_{\text{ex}}, \mathbf{v}) - b(\mathcal{R}_h^{n+1} \mathbf{u}_{\text{ex}}, q). \end{aligned}$$

As done in the previous labs, we can associate each variational form to a matrix/vector, representing the application of such forms to the basis functions $\{\varphi_i\}_{i=1}^{N_u}, \{\psi_l\}_{l=1}^{N_p}$ of the discrete velocity and pressure spaces V_h, Q_h , respectively:

$$A_{ij}^{\text{FE}} = m(\varphi_j, \varphi_i), \quad B_{li} = b(\varphi_i, \psi_l), \quad (\mathbf{F}^n)_i = F_{\text{FE}}^n(\varphi_i).$$

The algebraic system to be solved at each time step is, therefore:

$$\text{Find } \begin{bmatrix} \mathbf{U}^{n+1} \\ \mathbf{P}^{n+1} \end{bmatrix} \text{ s.t. } \begin{bmatrix} A^n & B^T \\ B & 0 \end{bmatrix} \begin{bmatrix} \mathbf{U}^{n+1} \\ \mathbf{P}^{n+1} \end{bmatrix} = \begin{bmatrix} \mathbf{F}^n \\ \mathbf{0} \end{bmatrix},$$

which can be solved with the same methods seen so far for the steady Stokes problem. Notice that the functional $\mathbf{F}^{n+1}$ depends on n because of its dependence on the solution $\mathbf{u}_h^n$ at the previous time step and on the source and boundary conditions functions $\mathbf{f}^n, \mathbf{u}_{\text{ex}}^{n+1}$.

2 - 3 We introduce also the backward Euler method (with a semi-implicit treatment of the advection term), so that we can discuss the implementation of the methods in a unified framework.

The semi-implicit backward Euler method with a semi-implicit treatment of the advection term can be formulated (in strong form) as:

Given $\mathbf{u}^0 = \mathbf{0}$, for each $n = 0, \dots, M-1$ find $(\mathbf{u}^{n+1}, p^{n+1})$ such that

$$\begin{cases} \frac{\mathbf{u}^{n+1} - \mathbf{u}^n}{\Delta t} + (\mathbf{u}^n \cdot \nabla) \mathbf{u}^{n+1} - \frac{1}{\text{Re}} \Delta \mathbf{u}^{n+1} + \nabla p^{n+1} = \mathbf{f}^{n+1}, & \text{in } \Omega, \\ \text{div } \mathbf{u}^{n+1} = 0, & \text{in } \Omega, \\ \mathbf{u}^{n+1} = \mathbf{u}_{\text{ex}}^{n+1}, & \text{on } \partial\Omega. \end{cases} \quad (3)$$

Correspondingly, the fully discrete problem reads as follows:

Given $\mathbf{u}_h^0 \simeq \mathbf{u}^0 = \mathbf{0}$, for each $n = 0, \dots, M-1$, find $(\mathbf{u}_h^{n+1}, p_h^{n+1}) \in V_{D,h}^{n+1} \times Q_h$, where $\mathbf{u}_h^{n+1} = \tilde{\mathbf{u}}_h^{n+1} + \mathcal{R}_h^{n+1} \mathbf{u}_h$ and $\tilde{\mathbf{u}}_h^{n+1} \in V_{0,h}$, such that

$$a_{\text{BE}}^{n+1}(\tilde{\mathbf{u}}_h^{n+1}, \mathbf{v}_h) + b(\mathbf{v}_h, p_h^{n+1}) + b(\tilde{\mathbf{u}}_h^{n+1}, q_h) = F_{\text{BE}}^{n+1}(\mathbf{v}_h, q_h) \quad \forall (\mathbf{v}_h, q_h) \in V_{0,h} \times Q_h, \quad (4)$$

where

$$\begin{aligned} a_{\text{BE}}^{n+1}(\mathbf{u}, \mathbf{v}) &= \left(\frac{1}{\Delta t} \mathbf{u}, \mathbf{v} \right) + ((\mathbf{u}_h^n \cdot \nabla) \mathbf{u}, \mathbf{v}) + \frac{1}{\text{Re}} (\nabla \mathbf{u}, \nabla \mathbf{v}), \\ b(\mathbf{v}, q) &= -(\text{div } \mathbf{v}, q), \\ F_{\text{BE}}^{n+1}(\mathbf{v}, q) &= \frac{1}{\Delta t} (\mathbf{u}_h^n, \mathbf{v}) + (\mathbf{f}^{n+1}, \mathbf{v}) - a_{\text{BE}}^{n+1}(\mathcal{R}_h^{n+1} \mathbf{u}_{\text{ex}}, \mathbf{v}) - b(\mathcal{R}_h^{n+1} \mathbf{u}_{\text{ex}}, q). \end{aligned}$$

The algebraic system associated to one step of the method reads as follows:

$$\text{Find } \begin{bmatrix} \mathbf{U}^{n+1} \\ \mathbf{P}^{n+1} \end{bmatrix} \text{ s.t. } \begin{bmatrix} A^{n+1} & B^T \\ B & 0 \end{bmatrix} \begin{bmatrix} \mathbf{U}^{n+1} \\ \mathbf{P}^{n+1} \end{bmatrix} = \begin{bmatrix} \mathbf{F}^{n+1} \\ \mathbf{0} \end{bmatrix},$$

where

$$A_{ij}^{n+1} = a_{\text{BE}}^{n+1}(\varphi_j, \varphi_i), \quad B_{li} = b(\varphi_i, \psi_l), \quad (\mathbf{F}^{n+1})_i = F_{\text{BE}}^{n+1}(\varphi_i).$$

Notice that the bilinear form a_{BE}^{n+1} and the functional $\mathbf{F}^{n+1}$, as well as the matrices and vector associated to them, depend on n because of their dependence on the solution $\mathbf{u}_h^n$ at the previous time step and on the source and boundary conditions functions $\mathbf{f}^{n+1}, \mathbf{u}_{\text{ex}}^{n+1}$.

From now on, we will use the notation of the BE method, which is more general since it includes also the time dependence of the matrix A .

The file `TEMPLATE_CFDlab06.ipynb` gives suggestions on how to structure the code: here we discuss its different components.

- The function `ex1_assemble_Euler(V, Q, Re, n, dt, t, u_old, u_ex, p_ex, f, create_pc_form=None, get_nsp=False)` is used to define the forms/functional and matrices/vectors associated to a single timestep of the fully discrete problem (4). The function's input arguments are the function spaces, the data of the problem, the discretization parameters, and the solution $\mathbf{u_old} = \mathbf{u}_h^n$ at the previous timestep. The exact solution and source functions are assumed to be passed as functions depending both on space and time (see next point). The function returns the `Function` `wh` that will contain the solution, the `LinearVariationalProblem` `vpb` to obtain it, and possibly the null space `nsp`, if required.

We also point out that the implementation of the advection term makes use of the `dot` operator to apply the tensor $\nabla \mathbf{u}$ to the linearized advecting velocity $\mathbf{u}_h^n$ (`u_old` in the code): indeed, $\mathbf{u}_h^n \cdot \nabla \mathbf{u} = (\nabla \mathbf{u}) \mathbf{u}_h^n$, where in right-hand side of the identity we mean standard matrix-vector product.

- Among the data, the exact solution and the source function f depend on both space and time. In this case, these functions are variable-separable, that is

$$f(\mathbf{x}, t) = f_1(\mathbf{x})f_2(t),$$

and we can exploit this in our Firedrake implementation. First, we define the `Constant` objects `sine_2_t`, `cosine_2_t`, corresponding to $\sin(2t)$, $\cos(2t)$, respectively: indeed, these are functions of time but **constant in space**. Then, we use them to build up $f_2(t)$ (and the corresponding time-dependent parts of $u_{\text{ex}}, p_{\text{ex}}$) and introduce the expressions `u_ex`, `p_ex`, `f` as functions of space and time. In this way, we will be able to update time evaluation in `u_ex`, `p_ex`, `f` just by acting on `sine_2_t`, `cosine_2_t`, without having to re-define the whole data functions, that would require to re-define also the variational problem `vpb`.

Indeed, we can define once and for all, even before starting the time loop cycle, the variational problem by calling once `ex1_assemble_Euler`.

- The function `solve_time_dependent(u_ex, p_ex, f, sine_2_t, cosine_2_t, t0, T, wh, vpb, nsp, params, u0)` implements the time-loop to solve the problem, taking as input arguments the variational problem and the variable-separated contributions to the data, as well as the parameters for the linear solver. We point out that at each time step, the data are updated by simply assigning a new value to `sine_2_t`, `cosine_2_t` (notice that we use `assign`, NOT the operator `=`): all the objects depending on them (like `u_ex` but also `vpb`) are automatically updated as a consequence. Then, linear variational solver is re-defined and then solved at each time: this triggers the re-assembling of the variational problem, namely the re-assembling of the matrices and vectors corresponding to the new problem to be solved at the current timestep.

The last ingredient to introduce to solve the problem is the preconditioner. We have seen in previous labs that a possible good preconditioner $P_{\Sigma^{n+1}}$ for the monolithic Stokes matrix

$$\Sigma^{n+1} = \begin{bmatrix} A^{n+1} & B^T \\ -B & 0 \end{bmatrix} \text{ has the form}$$

$$P_{\Sigma^{n+1}} = \begin{bmatrix} P_{A^{n+1}} & 0 \\ -B & P_S \end{bmatrix},$$

where $P_{A^{n+1}}, P_{S^{n+1}}$ are good preconditioners of A^{n+1} and $S^{n+1} = B(A^{n+1})^{-1}B^T$, respectively. We can take $P_{A^{n+1}} = A^{n+1}$ (or rather, its LU factorization), while for $P_{S^{n+1}}$ we have to consider that we are solving a **time-dependent** problem. A good preconditioner for the Schur complement in this case is the Cahouet-Chabard preconditioner P_{CC} , defined such that $P_{CC}^{-1} = (\nu M_p^{-1} + \frac{1}{\Delta t} K_p^{-1})^{-1}$. However, for ease of implementation, here we employ $P_* = \frac{1}{\nu} M_p + \Delta t K_p$ (where $1/\nu = \text{Re}$ in the problem at hand), which can be defined via a variational form and thus complies with the structures introduced in Lab 5: this preconditioner, although not certificated as optimal with respect to $h, \Delta t$, has the same scaling of P_{CC} with respect to ν and Δt , and thus it is euristically shown to be robust with respect to viscosity and time discretization.

Remark. Notice that the velocity-velocity block of the preconditioner contains **all velocity-velocity terms** of the original problem: e.g. it includes also the transport term in the backward Euler case.

Applying this solver in the forward Euler case yields absolutely unstable results: the norm of the solution is constantly increasing along time evolution, overcoming the value 1 that is an upper bound for $\|\mathbf{u}_{\text{ex}}\|_{L^2}$ at all times, and even exploding when many time steps are performed. Indeed, treating explicitly the viscous term induces a condition on the timestep which is extremely restrictive, thus making it computationally almost unfeasible. The backward Euler scheme, on the other hand, exhibits its (unconditionally) absolutely stable nature, with the numerical solution following accurately the exact solution. Absolute stability is preserved (with the current choice of Δt) if we treat the advection term in an *explicit* way (i.e. with $-((\mathbf{u}_h^n \cdot \nabla) \mathbf{u}_h^n, \mathbf{v}_h)$ at the right-hand side): indeed, the CFL condition associated to the advection term is way less restrictive than the condition associated to the viscous term.

- 4 To perform a timestep-convergence analysis, we wrap the implementation of the initial condition and the **for** loop described at the end of Exercise 1 into an additional **for** loop, accounting for different choices of Δt . In particular, we create a list **dt_vec** containing different values of Δt using Python's *list comprehension* syntax:

```
dt_vec = [0.04 * 2**ii for ii in range(0, -5, -1)]
```

with a *backward-step* use of the command **range**: **range**(0, -3, -1) $\rightarrow$ [0,-1,-2,-3] (the endpoint -4 is excluded).

For each value of Δt , we solve the problem, compute the error norms (accounting for pressure average correction as discussed in previous labs) and collect these errors in the lists **err_u_LinfL2**, **err_u_LinfH1**, **err_p_LinfL2**. Then, after the end of the outer **for** loop, we plot the errors against Δt , on a (bi)logarithmic scale: if an absolutely stable method is employed, the plot confirms the convergence order $\mathcal{O}(\Delta t)$ expected for the Euler method (cf. Figure 1).

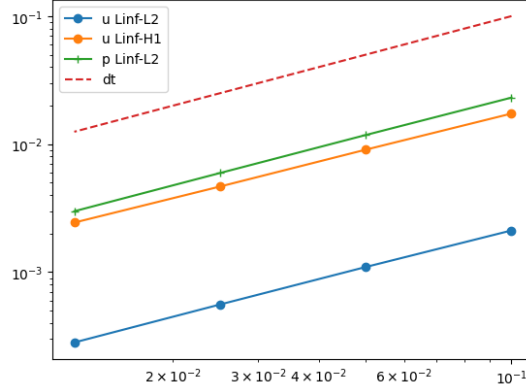


Figure 1: Computed errors of the Euler method against Δt , for a mesh having $n = 20$ subdivisions along each direction.

5

$$m(\tilde{\mathbf{u}}_h^{n+1}, \mathbf{v}_h) + b(\mathbf{v}_h, p_h^{n+1}) + b(\tilde{\mathbf{u}}_h^{n+1}, q_h) = F_{\text{FE}}^n(\mathbf{v}_h) \quad \forall (\mathbf{v}_h, q_h) \in V_{0,h} \times Q_h, \quad (5)$$

where

$$m(\mathbf{u}, \mathbf{v}) = \left(\frac{1}{\Delta t} \mathbf{u}, \mathbf{v} \right),$$

$$b(\mathbf{v}, q) = -(\operatorname{div} \mathbf{v}, q),$$

$$F_{\text{FE}}^n(\mathbf{v}) = m(\mathbf{u}_h^n, \mathbf{v}) + (\mathbf{f}^n, \mathbf{v}) - ((\mathbf{u}_h^n \cdot \nabla) \mathbf{u}_h^n, \mathbf{v}) - \frac{1}{\operatorname{Re}} (\nabla \mathbf{u}_h^n, \nabla \mathbf{v})$$

$$- m(\mathcal{R}_h^{n+1} \mathbf{u}_{\text{ex}}, \mathbf{v}_h) - b(\mathcal{R}_h^{n+1} \mathbf{u}_{\text{ex}}, q_h).$$

In the two-step BDF method, the partial time derivative is approximated by the following formula:

$$\frac{\partial \mathbf{u}}{\partial t} \simeq \frac{a_{-1} \mathbf{u}^{n+1} + a_0 \mathbf{u}^n + a_1 \mathbf{u}^{n-1}}{\Delta t} \quad (6)$$

where $a_{-1} = \frac{3}{2}, a_0 = -2, a_1 = \frac{1}{2}$. The linear extrapolation of the advection field, instead, is

$$\mathbf{u}^{n+1} \simeq b_0 \mathbf{u}^n + b_1 \mathbf{u}^{n-1}, \quad (7)$$

where the choice of $b_0 = 2, b_1 = -1$ ensures that such extrapolation is of the second order w.r.t. Δt . The resulting time-advancing scheme is:

Given $\mathbf{u}_h^0 \simeq \mathbf{u}^0 = \mathbf{0}$ and $\mathbf{u}_h^1$ obtained from one step of the Euler method, for each $n = 1, \dots, M-1$, find $(\mathbf{u}_h^{n+1}, p_h^{n+1}) \in V_{D,h}^{n+1} \times Q_h$, where $\mathbf{u}_h^{n+1} = \tilde{\mathbf{u}}_h^{n+1} + \mathcal{R}_h^{n+1} \mathbf{u}_h$ and $\tilde{\mathbf{u}}_h^{n+1} \in V_{0,h}$, such that

$$a_{\text{BDF2}}^{n+1}(\mathbf{u}_h^{n+1}, \mathbf{v}_h) + b(\mathbf{v}_h, p_h^{n+1}) + b(\mathbf{u}_h^{n+1}, q_h) = F_{\text{BDF2}}^{n+1}(\mathbf{v}_h, q_h) \quad \forall (\mathbf{v}_h, q_h) \in V_{h,0} \times Q_h, \quad (8)$$

where

$$a_{\text{BDF2}}^{n+1}(\tilde{\mathbf{u}}, \mathbf{v}) = \frac{a_{-1}}{\Delta t} (\mathbf{u}, \mathbf{v}) + ((b_0 \mathbf{u}_h^n \cdot \nabla) \mathbf{u}^*, \mathbf{v}) + ((b_1 \mathbf{u}_h^{n-1} \cdot \nabla) \mathbf{u}^*, \mathbf{v})$$

$$+ \frac{1}{\operatorname{Re}} (\nabla \mathbf{u}, \nabla \mathbf{v}),$$

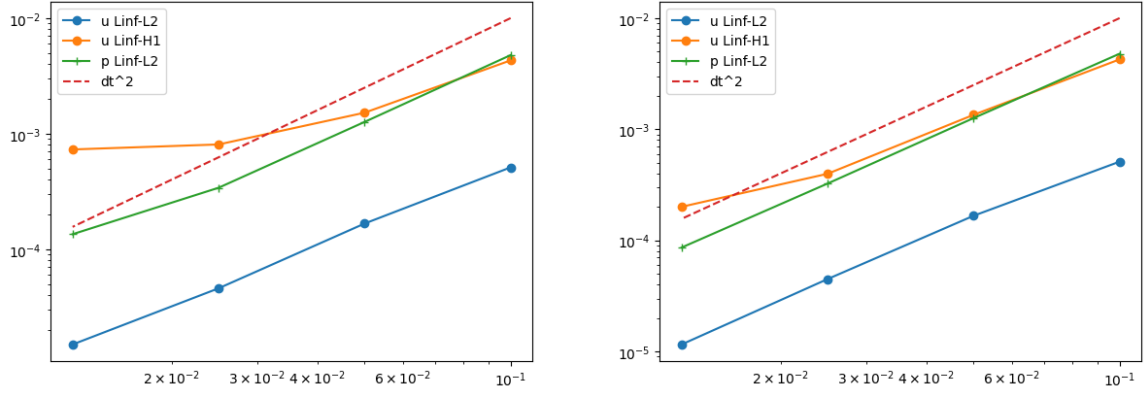


Figure 2: Computed errors of the BDF2 method against Δt , for a mesh having $n = 20$ (left) or $n = 30$ (right) subdivisions along each direction.

$$F_{\text{BDF2}}^{n+1}(\mathbf{v}, q) = -\frac{a_0}{\Delta t}(\mathbf{u}_h^n, \mathbf{v}) - \frac{a_1}{\Delta t}(\mathbf{u}_h^{n-1}, \mathbf{v}) + (\mathbf{f}^{n+1}, \mathbf{v}) - \frac{1}{\text{Re}}(\nabla \mathcal{R}_h^{n+1} \mathbf{u}_{\text{ex}}, \nabla \mathbf{v}) - b(\mathcal{R}_h^{n+1} \mathbf{u}_{\text{ex}}, q),$$

and the advected velocity $\mathbf{u}^*$ is equal to the solution velocity $\mathbf{u}_h^{n+1}$ in the semi-implicit case and equal to the extrapolated velocity $b_0 \mathbf{u}_h^n + b_1 \mathbf{u}_h^{n-1}$.

To implement the BDF2 method, we introduce a function `ex1_assemble_BDF2` that differs from `ex1_assemble_Euler` by these two features:

- in addition to the solution $\mathbf{u}_h^n$ obtained at the previous time step, passed as the input argument `u_old`, the function takes also $\mathbf{u}_h^{n-1}$ as an input, represented by `u_oldold`;
- the form `a`, the functional `L`, and the preconditioner form generator `create_pc_form` are modified in accordance with (8). In the attached notebook, both the semi-implicit and the explicit treatment of the advection term are reported: uncomment alternatively the one that you want to use.

Then, to perform the convergence analysis, we duplicate the code of point 4 into a new cell, with the following modifications:

- A two-steps initialization is considered, storing $\mathbf{u}^0$ in `u_oldold` and the solution of 1 step of the Euler method in `u0`.
- Accordingly, the inner `for` loop for time advancement starts from $t = 2\Delta t$.

The convergence plots are reported in Figure 2. From the plot on the right, we can see that the error decreases with Δt , but it seems to stagnate for small values of Δt . This is due to the choice of the mesh size $h = 1/n$, which is relatively coarse and thus saturates the space-time error when time discretization is sufficiently fine. To observe the expected convergence order, we need to consider a finer mesh, so that the approximation error in space is always lower than the one in time. In this case, choosing $n = 30$ is enough (see Figure 2 - right).